Sim-to-Real Joint Tracking with PPO

Presenter script, repeatable demo checklist, and Q&A

Voice: a mechatronics engineering student presenting a measured classroom experiment. Suggested delivery: approximately 15 minutes plus discussion; slide 17 is the source list and slide 18 is a backup. Use PowerPoint Presenter View for the same slide-by-slide notes.

The central claim I can defend

For this pair of PPO policies, domain randomization improved performance in one held-out simulated condition: success increased from 14/30 to 29/30, and mean episode RMSE decreased from 0.3299 to 0.2250 rad. Both achieved 30/30 nominal success. This is an educational simulation study, not a physical robot trial or full quadrupedal locomotion result.

What to keep separate

- Original evidence: 30 distinct matched evaluation seeds per policy and condition; one training seed per policy.
- Quick classroom demo: 3 seeds per condition, repeated twice. Expected held-out counts for these 3 seeds: fixed 1/3, randomized 2/3. This subset does not replace the 30-trial results.
- Repeated rounds demonstrate deterministic replay. They are not independent replications and do not increase the scientific sample size.
- Full retraining is a separate command. The quick replay only loads saved models and performs inference/evaluation.
- Slides show the verified cloud results. Locally regenerated values may differ slightly across platforms; identify any replacement data honestly.

Presentation plan

Slides 1–8: question, control loop, model, and experiment. Slides 9–12: evidence. Slide 13: live replay. Slides 14–16: validation, limitations, and conclusion. Slide 17: references. Slide 18: detailed results for questions.

Before class: Windows / VS Code checklist

1. Extract the entire classroom package. Keep main source, requirements, both policy ZIPs, and classroom/ together.
2. In VS Code, use File → Open Folder and select the folder containing reading5_sim2real_ppo.py. Opening only the Python file can prevent interpreter paths from resolving.
3. Use Python 3.12 and the project's .venv. Python 3.14 is not the tested interpreter for these pinned requirements.
4. Use Python: Select Interpreter and browse to the full path of .venv\Scripts\python.exe. The status bar should show Python 3.12.
5. Rehearse the replay with the projector connected. Open the deck and generated tracking PNG once before class. Installed dependencies and saved policies allow an offline demo.

One-time setup, from the project folder:

```
py -3.12 -m venv .venv

.\.venv\Scripts\python.exe -m pip install torch==2.7.1 --index-url https://download.pytorch.org/whl/cpu

.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

If Python 3.12 is absent: winget install --exact --id Python.Python.3.12, then restart VS Code. Package installation needs Internet access; the prepared demo does not.

Live run: default short demonstration

.\.venv\Scripts\python.exe classroom\demo.py --trials 3 --rounds 2 --open-figure

Alternatively, double-click classroom\run_demo.cmd. It uses the project interpreter explicitly. The supplied Windows launcher and image-viewer opening were not executed on the Linux cloud; the Python replay itself was executed successfully.

If something interrupts the demonstration

Missing gymnasium or torch: check the selected interpreter and install requirements into that same .venv. Missing policy ZIP: extract the complete package. No image window: open the PNG at the printed path. For a persistent machine problem, return to the embedded result slides and say, ‘These are saved outputs of the verified run; I am switching to a walkthrough of those results.’ Do not describe cached figures as a new live run.

Slide-by-slide presenter script

Read the script as needed, point to the indicated visual, and use the transition to keep the story connected. The short SAY phrase on each slide is a speaking anchor, not additional experimental evidence.

01. Can varied training physics improve transfer?

Suggested time: 40 seconds. Speaking anchor: “My question is whether varied training physics helps a controller handle an unfamiliar simulated joint.”

READ: For my Reading 5 project, I investigated a practical question: can a learned controller become more robust when its training simulation changes from episode to episode? I built a one-degree-of-freedom joint inspired by the control problems inside quadrupedal robots such as Go2 and ANYmal. Two PPO policies choose target angles for the same PD controller. One trains with fixed physics, and the other trains with randomized physics. I then compare them under nominal and unfamiliar simulated conditions. This is an educational simulation study, not a physical robot experiment.

POINT TO / DO: Point to the project question, then the fixed-versus-randomized comparison.

TRANSITION: First, I will connect that question to the mechanical, electrical, and control systems in a robot.

02. Why this is a mechatronics problem

Suggested time: 45 seconds. Speaking anchor: “A useful controller must work with the motor, the sensors, and the mechanical system together.”

READ: As a mechatronics student, I see the reality gap as an interaction between the mechanical plant, actuator behavior, sensing, and software. A heavier effective joint responds differently to torque. Friction changes damping, actuator latency delays corrections, and measurement noise distorts feedback. A policy trained against only one ideal combination can depend too strongly on that combination. My simplified model makes these effects visible without needing a complete walking robot. The engineering motivation is to develop a controller that tolerates uncertainty, while being honest about which physical effects the demonstration leaves out.

POINT TO / DO: Trace the mechanical, actuator, sensor, and software blocks.

TRANSITION: The control architecture separates the policy's target decision from the actuator torque calculation.

03. PPO chooses a target; PD supplies torque

Suggested time: 55 seconds. Speaking anchor: “The policy decides where to move; the PD controller computes the torque needed to move there.”

READ: The policy action is a target joint angle bounded between minus one and plus one radian. That target passes through a delay queue before reaching the PD controller. The controller uses proportional position error and derivative velocity feedback, then clips commanded torque to plus or minus twelve newton metres. The dynamics update the true joint state, and noisy measurements return to both the policy and PD feedback. This structure connects reinforcement learning to a familiar control loop. In this educational code, both the policy and PD calculation update every fifty milliseconds; I am not modeling a separate high-frequency motor controller.

POINT TO / DO: Follow the target-angle arrow, delay queue, PD equation, and feedback loop.

TRANSITION: Next, I will show exactly what the environment observes and how its dynamics advance.

04. A transparent Gymnasium joint environment

Suggested time: 50 seconds. Speaking anchor: “The model is small enough that we can inspect every observation, force balance, and integration step.”

READ: The environment observation contains measured angle, measured angular velocity, the current reference, the reference one hundred fifty milliseconds ahead, and the latest policy target. The preview is allowed because this is a prescribed trajectory, not an unknown future sensor measurement. Dynamics follow inertia times acceleration equals actuator torque minus viscous friction. Semi-implicit Euler updates velocity first, then angle using the new velocity. An episode lasts one hundred steps, or five simulated seconds, unless position or velocity exceeds the stated safety limits. Gymnasium reset and step methods are checked with Stable-Baselines3 before training.

POINT TO / DO: Point to the observation vector, Euler update, and terminated-versus-truncated conditions.

TRANSITION: Those states and torques also define the tracking reward and the separate success criterion.

05. Reward encourages tracking; success is predefined

Suggested time: 55 seconds. Speaking anchor: “The reward guides learning; the fixed RMSE threshold judges the final behavior.”

READ: The step reward starts at one and subtracts three costs. Five times squared tracking error emphasizes accuracy. The squared-velocity penalty discourages unnecessarily fast or oscillatory motion, and the squared-torque penalty discourages control effort. I score these terms using the true simulated state, while the policy sees noisy measurements. Success is defined separately: complete the full five-second episode without safety termination and achieve RMSE strictly below zero point three two radians. That threshold came from the original program and was retained before training. It is an educational tolerance, approximately eighteen degrees, rather than a real robot accuracy specification.

POINT TO / DO: Identify each reward term, then underline the strict RMSE < 0.32 rad condition.

TRANSITION: To make the comparison meaningful, both policies must share the learning setup and scoring rules.

06. Matched PPO training conditions

Suggested time: 55 seconds. Speaking anchor: “I changed the training physics distribution while keeping the other experimental choices consistent.”

READ: Both controllers use Stable-Baselines3 PPO with the same network architecture, seed, reward, reference-trajectory sequence, and training budget. Each policy receives twelve thousand two hundred eighty-eight environment steps, exactly forty-eight PPO rollouts of two hundred fifty-six steps. The hidden-layer architecture is thirty-two by thirty-two, with identical learning rate, batch size, update epochs, and discount factor. PPO is a particular actor-critic policy-gradient method using a clipped surrogate learning objective. It is not synonymous with every policy-gradient method, and its learning-objective clipping is different from the torque saturation inside the joint environment.

POINT TO / DO: Point across the matched settings, then distinguish PPO clipping from actuator saturation.

TRANSITION: The intended difference is the physical parameter distribution seen during training.

07. Nominal, randomized, and held-out physics

Suggested time: 55 seconds. Speaking anchor: “Randomization covers varied training joints; the held-out joint deliberately exceeds those ranges.”

READ: Fixed training uses inertia scale one, viscous friction zero point two, and no target-command delay. Randomized training samples inertia between zero point seven and one point eight, friction between zero point one and zero point six five, and delay from zero through three control steps at every reset. These values stay constant within that episode. The held-out condition uses inertia one point nine five, friction zero point seven five, and four delay steps, or two hundred milliseconds. It also increases position and velocity measurement noise. Both training modes use the same smaller sensor-noise levels.

POINT TO / DO: Compare each held-out value with the corresponding upper training boundary.

TRANSITION: Those experimental choices map directly onto a small set of functions and saved deliverables.

08. From readable code to auditable deliverables

Suggested time: 40 seconds. Speaking anchor: “The files connect environment design, actual learning, evaluation, and evidence that another student can check.”

READ: The source is organized around a few functions. Joint defines the environment, train creates each PPO policy, roll records an evaluation episode, summarize computes statistics, and figures creates the three requested plots. The trained policy files allow evaluation without repeating learning. Training rewards, episode records, and step records preserve the underlying numerical evidence. Requirements and the README explain installation and limitations. The executable is two hundred eighteen lines because it also includes complete plotting and audit exports. I chose inspectable output over compressing those functions into a difficult-to-read one hundred fifty lines.

POINT TO / DO: Point from each function name to its corresponding model, CSV, or figure output.

TRANSITION: The first output to inspect is the actual reward history recorded during training.

09. What the recorded training rewards show

Suggested time: 45 seconds. Speaking anchor: “These are actual training returns; the randomized policy is learning across a harder range of conditions.”

READ: This figure uses recorded completed-episode rewards, not constructed curves. Faint lines show individual episodes, and bold lines show a trailing ten-episode average. Both policies improve substantially during the short training budget. The randomized policy generally has lower training returns because its episodes span harder and more varied physical conditions. That does not automatically mean worse transfer, because the two training distributions differ. The meaningful comparison comes when both trained policies face the same evaluation conditions. A rising curve also does not establish full convergence; this is deliberately short CPU training for a classroom-scale experiment.

POINT TO / DO: Identify the faint raw episodes and bold moving averages; compare the training distributions.

TRANSITION: To compare transfer fairly, I evaluate the saved policies with matched conditions and seeds.

10. Evaluation: 30 matched trials per condition

Suggested time: 50 seconds. Speaking anchor: “The policies face matching trajectories and initial conditions, and every trial is saved.”

READ: Evaluation uses deterministic actions from each trained policy. I run thirty distinct matched seeds in nominal simulation and thirty in the held-out simulation for each policy, giving one hundred twenty episodes total. Separate random streams keep the reference and initial-state sequences matched despite physics randomization. The seeds also align underlying sensor-noise draws, although held-out noise is larger. I report mean episode reward, mean per-episode RMSE, sample standard deviations, and success percentages with Wilson intervals. These thirty evaluation trials vary conditions for already-trained policies; they are not thirty independent policy-training runs.

POINT TO / DO: Trace the two policies by two environments by thirty seeds design.

TRANSITION: One preselected held-out trial lets us see the tracking behavior behind those aggregate numbers.

11. Held-out tracking: one matched example

Suggested time: 50 seconds. Speaking anchor: “This preselected trial shows tracking behavior; the full thirty-trial evaluation supports the overall comparison.”

READ: Both panels show the same held-out physics and reference trajectory for seed one thousand, chosen before viewing the results. The fixed-physics policy has larger oscillation and overshoot, while the randomized policy follows this reference more closely. Higher inertia reduces acceleration for a given torque, friction changes damping, and delayed targets arrive after the state has changed. Their combined effect can produce lag and overshoot. Importantly, both policies pass the RMSE threshold in this particular trial: approximately zero point three zero eight and zero point one nine five radians. The full evaluation, not this one plot, determines the reported success rates.

POINT TO / DO: Compare identical dashed references, then point to the two displayed RMSE values.

TRANSITION: The grouped success chart summarizes all thirty paired trials rather than only this example.

12. Observed result: stronger held-out performance

Suggested time: 55 seconds. Speaking anchor: “Domain randomization helped this trained policy pair in the held-out simulation; it is not a universal guarantee.”

READ: Both policies succeed in all thirty nominal trials. In the held-out simulation, the fixed policy succeeds in fourteen of thirty trials, or forty-six point seven percent. The randomized policy succeeds in twenty-nine of thirty, or ninety-six point seven percent. Mean episode RMSE falls from zero point three two nine nine to zero point two two five zero radians. The error bars are ninety-five percent Wilson success intervals. Nominal RMSE is slightly higher for the randomized policy, so the result is not better accuracy everywhere. The supported conclusion is improved held-out robustness for this one trained policy pair.

POINT TO / DO: Point to nominal equality, held-out 14/30 versus 29/30, and the uncertainty bars.

TRANSITION: I will now replay the saved policies so the code-to-result connection is visible.

13. Live demonstration: replay, compare, repeat

Suggested time: 100 seconds. Speaking anchor: “I am reloading trained policies, evaluating matching trials, and checking repeatable execution—not training again.”

READ: For the live demonstration, I load the two saved model files and run the classroom helper with three trials and two rounds. The helper evaluates both policies in both conditions, then repeats exactly the same seeds to check reproducibility. Three-trial success rates can differ from the thirty-trial figures on the previous slide, so I will not substitute these quick-demo values for the scientific results. The second round is repeated execution, not independent evidence. Each invocation saves new timestamped output without changing the original models or report data. I can use thirty trials and one round to replay the complete evaluation.

POINT TO / DO: Switch to the prepared terminal, run --trials 3 --rounds 2, and show the repeatability check plus PNG.

TRANSITION: After the replay, I will summarize what was tested and how long the full experiment took.

14. Validation, provenance, and measured runtime

Suggested time: 45 seconds. Speaking anchor: “The plots are backed by raw records, executable checks, saved models, and measured wall-clock time.”

READ: The environment passes Stable-Baselines3's Gymnasium checker in all three physics modes. Eight regression tests cover seeding, dynamics, reward arithmetic, delay, episode handling, and safety limits. Seven artifact audits independently recompute statistics, check bounded controls and image quality, and reload the saved models to reproduce recorded trajectories. The verified full cloud run took twenty-seven point seven two seconds, including imports, both training runs, evaluation, and plotting; training took twenty point four five seconds. A repeat produced identical training and evaluation CSVs. Laptop timing and exact floating-point equality across platforms are not guaranteed.

POINT TO / DO: Point to the checker, 8 + 7 tests, raw-data trail, and full-run runtime label.

TRANSITION: Successful execution still leaves scientific and engineering limitations that matter for interpretation.

15. Limitations and the next engineering experiment

Suggested time: 55 seconds. Speaking anchor: “The next step is stronger evidence: more training seeds, classical baselines, and more realistic physics.”

READ: This model omits gravity, contacts, body coupling, balance, transmissions, and motor electrical or thermal behavior. Its inertia and noise are not calibrated to a real robot. Only one training seed was used, and evaluation uncertainty does not capture variability from retraining. The observation also hides the full delayed-action queue and physical parameters. Useful next experiments would add independent training seeds, a direct reference-following PD baseline, and one-parameter-at-a-time ablations. A richer simulator such as MuJoCo could add multibody and contact dynamics. Hardware testing would then require calibration, safety supervision, and evidence beyond this demonstration.

POINT TO / DO: Point first to missing physics, then the proposed baseline, seed, and ablation comparisons.

TRANSITION: My conclusion therefore needs to stay narrower than a claim of general real-world transfer.

16. Takeaway: robustness is an evidence-based claim

Suggested time: 35 seconds. Speaking anchor: “Randomization improved this simulated transfer test; the data define how far that conclusion can go.”

READ: My main takeaway is that the training distribution can matter as much as nominal training reward. In this experiment, varying inertia, friction, and target delay produced a policy with better held-out tracking and a higher success rate, while both policies retained full nominal success. I can demonstrate the comparison repeatedly by reloading the saved models, without confusing replay with new learning. The same workflow also preserves a disappointing result if randomization fails in a future experiment. For a mechatronics application, the value is a transparent path from assumptions to control behavior, numerical evidence, and clearly stated limitations.

POINT TO / DO: Return to the original question and point to the limited, evidence-supported answer.

TRANSITION: I have the reading connections and full numerical table available for questions.

17. How the four Reading 5 sources connect

Suggested time: 30 seconds. Speaking anchor: “The readings motivate the engineering choices; this small experiment does not reproduce their complete systems.”

READ: Tan and colleagues motivate attention to actuator behavior and randomized training physics in locomotion transfer. Zhao and colleagues place domain randomization within the broader reality-gap literature. Sutton and Barto's Chapter Thirteen explains policy-gradient and actor-critic foundations, while PPO is the later specific algorithm implemented here by Stable-Baselines3. MuJoCo illustrates the richer multibody, contact, constraint, and actuator physics omitted from my small Euler-integrated joint. Together, these sources explain why the experiment is relevant and why its limits matter. I am borrowing a research pattern, not claiming to reproduce a complete quadruped or its hardware performance.

POINT TO / DO: Point to each source's contribution and the full reference links.

TRANSITION: The final backup slide contains the complete statistics if a numerical detail is useful.

18. Backup: complete verified numerical comparison

Suggested time: 30 seconds. Speaking anchor: “These are the full thirty-trial statistics; quick replay values are labeled separately.”

READ: This table is the numerical reference for the presentation. Each row represents thirty episodes for one policy and one evaluation condition. Reward and RMSE are reported with sample standard deviations, while success includes a count, percentage, and Wilson interval. The randomized policy has slightly higher nominal mean RMSE despite similar reward and identical nominal success, because reward also includes speed and torque costs. Held-out performance improves across the reported tracking and success measures. These numbers come from the original verified evaluation CSVs; a three-trial classroom replay is a separate illustration with a smaller sample.

POINT TO / DO: Use the relevant row and column to answer a specific numerical question.

TRANSITION: I will return to the relevant slide for any questions about the model, learning method, or evidence.

Live-demo runbook

Mode	Command after the project interpreter	Meaning
Quick replay	classroom\demo.py --trials 3 --rounds 2	12 episodes per round; same 3 paired seeds; two identical rounds
Full evaluation replay	classroom\demo.py --trials 30 --rounds 1	120 episodes, original 30-seed evaluation design; no training
Full retraining	reading5_sim2real_ppo.py	Runs check_env, trains both models, evaluates, replaces root experiment outputs

Before pressing Enter

Say: ‘These policies are already trained. I am testing the saved controllers, not teaching them again.’

During execution

Point to policy/environment labels and the repeatability PASS message. Each simulated episode lasts five seconds, but simulation can run faster than wall-clock time.

When the plot opens

Identify dashed desired angle and actual angle. The first held-out example is seed 1000; both policies pass the threshold on this particular trial.

When repeating

Run the command again. A new timestamped folder prevents previous results being overwritten. Same-seed repeats demonstrate execution repeatability only.

When showing numerical results

The quick subset gives fixed 1/3 and randomized 2/3 held-out successes in the verified run. Refer back to the original 30-trial slide for 14/30 versus 29/30.

If you choose to retrain

Use a disposable copy of the complete project to preserve the prepared data. Report whatever that run produces; do not change the success threshold or select favorable seeds.

Measured cloud reference: full experiment 27.72 s, training 20.45 s. The tested short replay took 7.06 s, and full evaluation replay took 9.32 s, including imports. These timings are separate operations and vary across machines.

Questions and answers

Start with the short answer. Use the longer answer if the instructor asks for mechanics, learning, or experimental details.

1. Is this an actual physical sim-to-real robot experiment?

SHORT ANSWER: It is transfer between two simulated conditions, not hardware validation.

No physical robot was used. The nominal and held-out environments are two configurations of the same simplified joint simulator. Greater inertia, friction, delay, and noise create an unfamiliar simulated approximation of reality. This is useful for demonstrating a transfer hypothesis, but it does not establish real robot performance. Hardware validation would require a calibrated model, reliable sensing and actuation, safety supervision, and controlled physical trials.

Relevant slides: 1, 7, 15

2. Why model one joint instead of an entire quadruped?

SHORT ANSWER: One joint makes the learning and control effects inspectable within a short classroom runtime.

A one-degree-of-freedom model lets us isolate tracking, PD feedback, delay, inertia, and friction without the extra difficulty of balance or contact planning. It is fast enough to execute and inspect during class. The tradeoff is limited realism: the model cannot evaluate walking, ground contact, full-body coupling, foot placement, or stability of a quadruped. The Go2 and ANYmal connection is an engineering motivation, not a claim that this is their controller.

Relevant slides: 1, 2, 15

3. Why does the policy output an angle target rather than motor torque?

SHORT ANSWER: Target angles preserve an interpretable PD control structure while separating learned decisions from actuation.

The policy chooses a bounded target angle and the PD law converts tracking error and angular velocity into torque. This resembles a useful engineering separation between a learned command layer and a familiar feedback controller. It also makes saturation and delay easy to explain. However, that architecture does not guarantee safety or stability. Actual robot controllers often update their inner loops faster than this demonstration, where policy and PD both use a fifty-millisecond step.

Relevant slides: 3, 4

4. What makes PPO different from policy-gradient methods in general?

SHORT ANSWER: PPO is a specific actor-critic policy-gradient algorithm with a clipped surrogate learning objective.

Policy-gradient methods form a broad family that adjusts policy parameters to increase expected return. Sutton and Barto provide those foundations, including actor-critic ideas. PPO is a later algorithm that uses a particular surrogate objective involving action-probability ratios, with clipping intended to discourage excessively large updates. Stable-Baselines3 supplies that implementation here. The demonstration should therefore describe PPO as one policy-gradient method, rather than treating PPO and policy gradients as interchangeable terms.

Relevant slides: 6, 17

5. Does PPO clipping limit the joint torque?

SHORT ANSWER: No. PPO objective clipping and actuator torque saturation are separate mechanisms.

PPO clipping operates during learning on a ratio of action probabilities under the updated and previous policies. It is intended to restrain the policy update. Torque saturation is a separate physical-model rule: the PD command is clipped to plus or minus twelve newton metres at each step. The policy target is also bounded to plus or minus one radian. These three bounds have different roles and should not be described as the same safety mechanism.

Relevant slides: 3, 6

6. Is the observation fully Markov when actuator delay is present?

SHORT ANSWER: Not fully; delayed-command history and randomized physical parameters are partly hidden.

The observation contains noisy joint state, the current and preview reference, and the latest target, but not the entire pending command queue. It also omits the sampled inertia, friction, and delay. As a result, identical observations can correspond to different hidden dynamics or queued commands. This is a small partially observed control problem. Adding action-observation history, explicit parameter estimates, or a recurrent policy would be reasonable extensions; the present memoryless PPO comparison keeps both policies consistent.

Relevant slides: 4, 7, 15

7. Is providing the future reference unfair or a form of cheating?

SHORT ANSWER: The future desired trajectory is known and supplied equally to both policies; future measured state is not supplied.

The reference is a prescribed sinusoid, so its value one hundred fifty milliseconds ahead is available in advance. Giving a controller preview of a known desired command is different from revealing a future sensor measurement or future realized joint state. Both policies receive the same observation format and trajectory information. The limitation is the application assumption: this setup represents planned joint tracking, not an unknown or arbitrary reference that arrives without preview.

Relevant slides: 4, 6

8. Why can delay cause oscillation, and is this controller proven stable?

SHORT ANSWER: Delay applies old targets to a changing state; the experiment does not prove closed-loop stability.

The queue holds a target while the joint continues to move, so a correction may arrive after the state has changed substantially. That can increase lag or overshoot and interact with the feedback gains. The chosen gains, saturation, sampled dynamics, and safety limits were tested through the reported episodes, but successful episodes are not a formal stability proof. A rigorous analysis would need the delayed closed-loop dynamics and a clearly specified region of operation.

Relevant slides: 3, 7, 11, 15

9. What is the difference between changing inertia and changing friction?

SHORT ANSWER: Inertia scales acceleration; viscous friction produces velocity-proportional damping.

The modeled acceleration is net torque divided by joint inertia. A larger inertia therefore reduces acceleration for the same applied torque. Viscous friction subtracts a torque proportional to velocity, changing damping and the effort required during motion. Their effects are coupled with the PD gains and delay. Since all held-out changes are applied together, the current comparison measures their combined effect; separate ablations would be needed to attribute an outcome to a particular parameter.

Relevant slides: 2, 4, 7, 11

10. Why was success defined as RMSE below 0.32 rad?

SHORT ANSWER: It preserves the original predeclared educational threshold rather than tuning the criterion after seeing the results.

The threshold was retained from the uploaded program and written into the experiment plan before revised training. Success also requires the full five-second episode and no safety termination. Zero point three two radians is about eighteen point three degrees, so it is a generous educational tolerance rather than a hardware safety or precision standard. A real application would derive the limit from task requirements. Changing it after examining results would weaken the comparison.

Relevant slides: 5, 10, 12

11. What do the 95% success intervals tell us?

SHORT ANSWER: They describe evaluation success uncertainty for these trained policies, not uncertainty across independent training runs.

The plotted intervals are Wilson binomial intervals based on thirty successes or failures per policy and condition. They show that an observed percentage from a finite number of evaluation episodes has uncertainty. They do not establish a ninety-five percent probability of hardware success, and they do not incorporate the variability of training a new policy from another seed. Because policies share paired evaluation seeds, a direct confidence interval for their difference would also need an appropriate paired analysis.

Relevant slides: 10, 12, 18

12. Do thirty evaluation seeds mean the result was reproduced across thirty training runs?

SHORT ANSWER: No. There is one training seed per policy and thirty distinct evaluation trials per condition.

Both policies were trained with seed forty-two. The thirty evaluation seeds vary reference amplitude, frequency, phase, initial angle, and sensor-noise realization for those already-trained models. They test trial-to-trial performance, not variability from learning again. Repeating the saved-model demo also does not add training seeds. The next stronger experiment would train several independent policy pairs and summarize held-out performance across those pairs as well as across evaluation trials.

Relevant slides: 6, 10, 13, 15

13. Why can nominal reward improve while nominal RMSE becomes slightly worse?

SHORT ANSWER: Reward includes velocity and torque costs as well as tracking error.

Mean episode reward is not simply the negative of reported mean RMSE. It sums squared error, squared velocity, and squared torque penalties over time, while the table reports the mean of each episode's RMSE. A policy can therefore trade a small amount of tracking accuracy for smoother or less effortful motion and obtain a slightly higher return. Both nominal success rates are one hundred percent, so the small nominal difference should not be overstated.

Relevant slides: 5, 12, 18

14. What exactly is randomized, and where does the sensor noise enter?

SHORT ANSWER: Training randomizes inertia, friction, and delay at resets; noisy state feeds both policy and PD.

The randomized training environment draws inertia scaling, viscous friction, and an integer command delay at every episode reset, then holds them constant during that episode. Both training modes use the same small Gaussian position and velocity noise levels. The held-out environment uses larger noise. A noisy state measurement enters both the policy observation and PD feedback, while dynamics and scoring retain the simulator's true state. This distinction prevents confusing measurement quality with ground-truth tracking error.

Relevant slides: 3, 4, 7

15. Why use semi-implicit Euler, and what does the fifty-millisecond timestep imply?

SHORT ANSWER: It is simple and explicit to inspect, but timestep sensitivity and physical fidelity remain limitations.

Semi-implicit Euler first updates angular velocity using acceleration, then updates position using that new velocity. It is easy to implement and explain for this small demonstration. The fifty-millisecond timestep means twenty updates per simulated second for both policy and PD calculations. That is a modeling choice, not a claim of realistic motor-controller bandwidth. A stronger simulation study should examine timestep sensitivity and compare against smaller steps or a more capable physics engine.

Relevant slides: 3, 4, 15

16. Could an ordinary PD controller solve this task without reinforcement learning?

SHORT ANSWER: Possibly; a direct reference-following PD baseline is an important missing comparison.

A tuned conventional controller might track this simple sinusoidal joint task well. The current experiment compares fixed-physics PPO training with randomized-physics PPO training, while both use the same inner PD law. It does not compare learned control against a hand-designed baseline and therefore does not prove that RL is needed. Adding a policy-free PD controller, a robustly tuned version, and perhaps a model-based controller would clarify whether learning adds value beyond this distributional comparison.

Relevant slides: 3, 6, 15

17. Do repeated classroom rounds increase the scientific sample size?

SHORT ANSWER: No. The repeated rounds reuse the same seeds to demonstrate reproducible execution.

The quick helper uses three distinct seeds per policy and condition, then repeats those same seeds in its second round. Matching records demonstrate repeatability for the loaded policies, but repeated identical trials are not independent new evidence. Three-trial percentages may also differ noticeably from the thirty-trial project results. For the scientific comparison, retain the original thirty distinct matched trials. To replay that design, use thirty trials and one round rather than combining repeated quick-demo rounds.

Relevant slides: 10, 12, 13

18. How can I repeat the demo or show fresh training without losing the report results?

SHORT ANSWER: Use the replay helper repeatedly; perform full retraining in a disposable project copy.

The classroom helper loads saved policies and writes each invocation into a new timestamped demo_runs directory. It leaves the root models and report data unchanged. Use three trials and two rounds for a short reproducibility demonstration, or thirty trials and one round for the full evaluation design. The original reading5_sim2real_ppo.py command retrains both policies and replaces its root outputs, so run that version in a separate disposable copy when preserving the prepared report evidence.

Relevant slides: 8, 13, 14

19. What should I do if VS Code reports missing gymnasium or an invalid interpreter path?

SHORT ANSWER: Open the project folder and select its Python 3.12 virtual-environment executable using the full path.

The class package expects its pinned dependencies in a Python three point twelve environment. A Python three point fourteen selection or another interpreter can report missing packages even after installation elsewhere. Open the actual project folder in VS Code, select its .venv Scripts python.exe using the full Windows path, and use that same executable for pip and execution. Before class, verify imports and run the helper. If execution fails, label the prepared figures as saved evidence rather than claiming a live run.

Relevant slides: 8, 13, 14

20. How does this relate to MuJoCo and a future real robot experiment?

SHORT ANSWER: MuJoCo can add richer physics; hardware transfer still requires calibration and independent validation.

MuJoCo provides multibody dynamics, contacts, constraints, and actuator models that the one-joint Euler simulation omits. It could support the next stage of a more realistic learning experiment, including body interaction and foot contact. That upgrade would not automatically establish hardware transfer. Tan's work motivates careful actuation and randomization, while Zhao's survey explains broader reality-gap strategies. A physical deployment would still require calibrated uncertainty ranges, sensing and latency measurements, protective limits, and supervised validation.

Relevant slides: 15, 17

Exact-results reference

Policy	Condition	Reward ± SD	RMSE ± SD (rad)	Success	95% Wilson
Fixed	Nominal	93.419 ± 2.645	0.1022 ± 0.0225	100.0%	88.6–100.0%
Fixed	Held-out	34.082 ± 26.052	0.3299 ± 0.0734	46.7%	30.2–63.9%
Randomized	Nominal	93.666 ± 2.621	0.1037 ± 0.0235	100.0%	88.6–100.0%
Randomized	Held-out	70.970 ± 12.653	0.2250 ± 0.0505	96.7%	83.3–99.4%

SD is the sample standard deviation of the 30 episode outcomes. RMSE is first computed separately per episode and then averaged. Wilson intervals describe evaluation success for fixed trained policies, not uncertainty across training seeds. The success rule also requires a full episode without safety termination.

Sources and how I connect them

Tan et al. (2018), Sim-to-Real: Learning Agile Locomotion for Quadruped Robots — https://arxiv.org/abs/1804.10332. Motivation: learned joint commands, actuation modeling, and randomized simulation.

Zhao et al., Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: A Survey — https://arxiv.org/abs/2009.13303. Motivation: the reality gap and limits of transfer strategies.

Sutton and Barto, Reinforcement Learning: An Introduction, Chapter 13 — http://incompleteideas.net/book/the-book-2nd.html. General policy-gradient and actor–critic foundations.

MuJoCo — https://mujoco.org/. A richer multibody/contact simulator; not a dependency of this example.

Schulman et al. (2017), Proximal Policy Optimization Algorithms — https://arxiv.org/abs/1707.06347. The specific clipped-surrogate PPO method, distinct from policy gradients in general.

Evidence files: metrics.csv, evaluation_episodes.csv, evaluation_steps.csv, training_rewards.csv, experiment.json, verification.txt, environment_tests.txt, and classroom/demo_validation.json. The older uploaded ZIP's custom-PPO results are not the source of this deck's SB3 claims.